



LLM Interview Questions & Answers

100+ questions with detailed answers



Kalyan KS

Q1. CNNs and RNNs don't use positional embeddings. Why do transformers use positional embeddings?

CNNs and RNNs don't use positional embeddings because these models inherently capture positional information through convolutional filters or recurrence over time steps. However, the Transformer model uses the self-attention mechanism, which processes all tokens in parallel without any notion of sequence or order.

Positional embeddings inject explicit information about each token's position in the sequence, allowing transformers to understand word order and relative distances between tokens. This helps them model sequential dependencies and sentence structure effectively, compensating for the parallel and order-agnostic nature of self-attention mechanisms.

Q2. Tell me the basic steps involved in running an inference query on an LLM.

Running an inference query on an LLM involves the following steps:

Tokenization: The input text is first broken down into tokens, and then the tokens are mapped to their IDs in the model's vocabulary.

Prefill Phase: The embedding layer converts these token IDs into embeddings, and then the decoder layers process these embeddings simultaneously to compute intermediate states (keys and values). This parallel processing establishes the context for generating new tokens.

Decoding Phase: The model generates output tokens one at a time autoregressively, each based on previously generated tokens and cached states, continuing until a stopping condition is met.

Detokenization: Finally, the generated tokens are converted back into human-readable text.

Q3. Explain how KV Cache accelerates LLM inference.

KV Cache speeds up LLM inference by storing the attention key (K) and value (V) representations that were computed for previous tokens. This allows the model to reuse them instead of recalculating them at each decoding step. During text generation, only the query for the latest token is computed and then combined with the cached K and V vectors to produce the next token, significantly reducing redundant computation.

This dramatically speeds up inference—often delivering several-fold faster token generation. However, the cache consumes substantial GPU memory, which is why techniques like KV cache offloading and compression are used to balance speed and memory efficiency in large-scale LLM serving.

Q4. How does quantization affect inference speed and memory requirements?

Quantization reduces the numerical precision of LLM's parameters, such as converting 32-bit floating-point weights to lower bit widths like 8-bit integers or 4-bit values. This compression dramatically decreases the model's memory footprint, often by 50-75% or more, enabling deployment on smaller devices and allowing more data to fit into cache memory.

Simultaneously, quantization speeds up inference because lower-precision operations require fewer computational resources and can be processed faster by modern hardware optimized for such formats.

However, this efficiency gain may come at a small cost to model accuracy, necessitating calibration or quantization-aware training to minimize performance degradation. Overall, quantization is a vital technique for reducing memory usage and improving inference speed in practical LLM applications.

Q5. How do you handle the large memory requirements of KV cache in LLM inference?

Handling large KV Cache memory requirements in LLM inference is crucial for high-throughput serving and long context windows, as the cache size grows linearly with sequence length and batch size. We can handle large memory requirements of KV Cache in LLM inference using techniques like PagedAttention, Grouped-Query, Multi-Query Attention, Quantization, or Cache Offloading.

PagedAttention manages the cache using fixed-size blocks like an operating system's virtual memory, reduces memory fragmentation, and enables higher batch sizes. Grouped-Query Attention (GQA) or Multi-Query Attention (MQA) decreases the size of the KV cache by reducing the number of Key/Value heads.

Finally, strategies such as quantizing the KV cache (e.g., to INT8) or offloading inactive cache data to cheaper storage (like CPU RAM) also help manage the memory footprint effectively.

Q6. After tokenization, how are tokens converted into embeddings in the Transformer model?

Tokens, represented as integer IDs after tokenization, are converted into embeddings using a lookup table called an embedding matrix. For every token in the model's vocabulary, this matrix has a dense, fixed-size vector (the embedding).

Specifically, the token ID is used as an index to retrieve its corresponding high-dimensional vector from the embedding matrix. This retrieved vector is the numerical representation that captures the token's semantic and syntactic meaning. The embeddings of input tokens are then processed by the subsequent transformer layers.

Q7. Explain why subword tokenization is preferred over word-level tokenization in the Transformer model.

Subword tokenization, such as Byte-Pair Encoding (BPE) in the Transformer model, offers several advantages over word-level tokenization by handling out-of-vocabulary words and balancing vocabulary size. It effectively handles out-of-vocabulary (OOV) words (unseen words, i.e., words not in the model vocabulary) by breaking them into known subwords, significantly improving coverage of rare or unseen words.

This method also results in a smaller, manageable vocabulary size compared to the vast lexicon of word-level tokenization, while still providing the semantic benefits of word-level units. Finally, it helps the model learn about morphology (prefixes, suffixes) by segmenting words into meaningful parts.

Q8. Explain the trade-offs in using a large vocabulary in LLMs.

Using a large vocabulary in LLMs allows the model to represent words more precisely and reduce token fragmentation, improving understanding of rare or domain-specific terms. However, it increases the size of the embedding and output layers, leading to higher memory usage and slower training.

Larger vocabularies also make softmax computations more expensive and may lead to a sparser representation of less frequent tokens, potentially hindering effective learning for those words. Therefore, models often moderate vocabulary sizes to ensure a balance between linguistic coverage and computational efficiency.

Q9. Explain how self-attention is computed in the Transformer model step by step.

Self-attention in the Transformer model is computed by first projecting each input token into three vectors: queries, keys, and values using learned weight matrices. Then, attention scores are calculated by taking the dot product of a query with all keys, scaled by the square root of the key dimension to stabilize gradients.

These scores are normalized using the softmax function to obtain attention weights, representing the importance of each token relative to others. Finally, each token's output is computed as the weighted sum of the value vectors, allowing the model to capture contextual relationships within the input sequence.

Q10. What is the computational complexity of self-attention in the Transformer model?

The computational complexity of the self-attention mechanism in the Transformer model is primarily quadratic with respect to the sequence length n . Formally, it is $O(n^2 \cdot d)$. The $O(n^2 \cdot d)$ comes from computing the attention scores between all pairs of tokens. Since there are $n \times n = n^2$ pairwise interactions, and each involves a dot product of d dimensional vectors, this results in $n^2 \cdot d$ operations.

The quadratic complexity with respect to sequence length n makes self-attention computationally expensive for long sequences. However, this design allows the model to capture relationships between all tokens simultaneously, which is crucial for its performance. Despite its quadratic complexity, it remains faster than traditional RNNs in practice due to parallelization benefits.

Q11. How do Transformer model address the vanishing gradient problem?

Transformers address the vanishing gradient problem primarily through the use of residual (skip) connections and layer normalization. Residual connections allow gradients to flow directly through the network by adding the input of a layer to its output, preventing gradients from shrinking during backpropagation.

Layer normalization helps stabilize training by keeping activations within a consistent range, which controls the scale of gradients. Additionally, the self-attention mechanism enables direct information flow between tokens, reducing the depth of dependency and further mitigating gradient vanishing.

Q12. What is tokenization, and why is it necessary in LLMs?

Tokenization is the process of converting raw text into smaller, discrete units called tokens, which can be words, sub-words, or characters. This step is necessary in LLMs because models cannot directly process raw strings of text; they require numerical input.

Tokenization allows mapping the input text to a vocabulary of known tokens. These tokens are then converted into numerical representations called embeddings. These embeddings are then used by LLMs for training and inference, fundamentally enabling input text processing.

Q13. Explain the role of token embeddings in the Transformer model.

Token embeddings in the Transformer model encode syntactic and semantic information. These embeddings are obtained from an embedding matrix, where each token's index maps to a fixed-size vector. These token embeddings are combined with positional embeddings to help the model retain word order. These enriched representations are then passed into the subsequent Transformer model layers, enabling context-aware understanding and text generation.

Q14. Explain the working of the embedding layer in the transformer model.

The main purpose of the embedding layer in the transformer model is to convert input token IDs into dense, continuous vectors called embeddings. Each token is first

represented as a one-hot vector and then multiplied with an embedding matrix to produce the token embedding.

These token embeddings are then combined with positional embeddings to retain information about token order within the sequence. To summarize, the embedding layer transform the sequence of tokens represented as integer IDs into the sequence of embeddings. These embeddings are then processed by the subsequent layers to inject contextual information.

Q15. What is the role of self-attention in the Transformer model, and why is it called “self-attention”?

The self-attention mechanism enables the model to compute context-rich representation for each token by allowing the tokens to attend to all tokens in the input sequence. These context-rich representations capture long-range dependencies and relationships.

The mechanism is called “self-attention” because the attention is computed within the same sequence - each token attends to itself and others without external input. This process helps build rich contextual embeddings crucial for understanding meaning and structure in text.

Q16. What is the purpose of the encoder in a transformer model?

The purpose of the encoder in a transformer model is to process the entire input sequence and generate context-aware numerical representations. The encoder uses multi-head self-attention and feed-forward neural networks to capture the relationships and dependencies between all the tokens to generate context-rich token representations. These context-rich token representations are then passed to the decoder, which generates the output sequence.

Q17. What is the purpose of the decoder in a transformer model?

The purpose of the decoder in a transformer model is to generate the output sequence based on the contextual representations provided by the encoder and the previously generated tokens.

The decoder layers use (i) masked self-attention to ensure that predictions for the current step rely solely on previously generated outputs and (ii) encoder-decoder

attention to focus on the most relevant parts of the encoder's final output when generating each output token.

In simple words, the decoder transforms the encoder's output (contextual representations) into the desired output token by token.

Q18. How does the encoder-decoder structure work at a high level in the Transformer model?

The encoder-decoder structure in a Transformer is used for sequence-to-sequence tasks like machine translation. At a high level, the encoder processes the entire input sequence simultaneously, using self-attention to create rich, context-aware vector representations (the encoded state).

The decoder then uses a separate self-attention mechanism, masked to prevent looking ahead, to generate the output sequence one token at a time, autoregressively. The decoder also incorporates an encoder-decoder attention layer, allowing it to focus on the most relevant parts of the encoder's output for each token it generates.

Q19. What is the purpose of scaling in the self-attention mechanism in the Transformer model?

The purpose of scaling in the self-attention mechanism of a Transformer model is to stabilize training by controlling the magnitude of the dot-product values between query and key vectors. Without scaling, the dot products can grow large in high-dimensional spaces, resulting in extremely small gradients that slow down or destabilize learning.

To prevent this, the dot products are divided by the square root of the key dimension, which keeps the attention scores in a range that leads to more effective gradient flow and stable optimization. Thus, the scaling step ensures that the self-attention mechanism can effectively focus on relevant tokens of the input sequence without numerical issues during training.

Q20. Why does the Transformer model use multiple self-attention heads instead of a single self-attention head?

Transformer models use multiple self-attention heads instead of a single one because they enable the model to capture diverse relationships and patterns in the input. This is similar to using multiple flashlights to illuminate different aspects of a scene.

Using a single head would restrict the model to one form of relationship, limiting its capacity to fully understand the input data. Multi-head attention, by allowing the model to capture diverse relationships, results in richer contextual representations and subsequently better performance across NLP tasks.



Support the Author

I hope you found this “LLM Interview Questions and Answers” book highly useful.

I’ve made this book freely available to help the AI and NLP community grow and to support learners like you. If you found it helpful and would like to show your appreciation, **you can buy me a coffee** to keep me motivated in creating more free resources like this.

👉 [Buy Me a Coffee](#)

Your small gesture goes a long way in supporting my work—thank you for being part of this journey! 🙏

— Kalyan KS

Q21. How are the outputs of multiple heads combined and projected back in the multi-head attention in the Transformer model?

In the multi-head attention model of the Transformer, the outputs from multiple self-attention heads are first computed independently, each producing a contextually weighted representation of the input in different subspaces. These outputs are then concatenated along the feature dimension to form a single combined vector.

This concatenated result is passed through a final linear projection layer, which projects it back into the model's expected dimension for further processing. This mechanism allows the model to integrate diverse information captured by each head into a unified representation.

Q22. How does masked self-attention differ from regular self-attention, and where is it used in a Transformer?

Masked self-attention differs from regular self-attention by restricting the attention mechanism so that each position in the sequence can only attend to earlier positions and itself, preventing access to future tokens. This is essential for autoregressive tasks like text generation, where tokens are generated sequentially one by one.

Regular self-attention allows every token to attend to all tokens in the sequence, which is useful in encoder layers for understanding context bidirectionally. Masked self-attention is specifically used in the decoder layers of the Transformer model to ensure that the model predicts tokens one at a time without peeking ahead.

Q23. Discuss the pros and cons of the self-attention mechanism in the Transformer model.

Self-attention enables the Transformer model to capture long-range dependencies (contextual relationships) between tokens in the input sequence efficiently. It allows for parallel computation, improving training speed compared to recurrent models.

However, its quadratic time complexity with respect to sequence length and large memory footprint make it computationally expensive for long inputs.

Q24. What is the purpose of masked self-attention in the Transformer decoder?

During output generation, masked self-attention in the decoder prevents the model from looking ahead by blocking access to future tokens during generation. This means that when predicting a token, the model is restricted to attending only to the tokens that come before it in the sequence.

This ensures (i) auto-regressive behavior, allowing the model to generate text one token at a time, and (ii) the model learns to make predictions based solely on past context, maintaining the integrity and correctness of language modeling.

Q25. Explain how masking works in masked self-attention in Transformer.

Masking in masked self-attention (used in the Transformer's decoder) prevents the model from attending to future tokens in the sequence. It's achieved by setting the attention scores for those future positions to a very large negative number (like $-\infty$), so that the softmax function turns them into zero. This ensures that the prediction for the current token only relies on the known preceding tokens, preserving the auto-regressive property.

Q26. Explain why self-attention in the decoder is referred to as cross-attention. How does it differ from self-attention in the encoder?

Self-attention in the decoder is referred to as cross-attention because it involves attending to the encoder's output while generating each token in the decoder. Specifically, in cross-attention, the queries come from the decoder's previous layer, while the keys and values come from the encoder's output, allowing the decoder to focus on relevant parts of the input sequence.

This differs from self-attention in the encoder, where the attention mechanism operates solely within the input sequence itself, with queries, keys, and values all derived from the same data. Thus, the encoder's self-attention captures internal dependencies, while the decoder's cross-attention links the decoder to the encoder representations for contextual generation.

Q27. What is the softmax function, and where is it applied in the Transformer model?

The softmax function is a mathematical operation that converts a vector of raw scores (logits) into a probability distribution, where each value lies between 0 and 1, and all values sum to 1. In the Transformer model, softmax is applied in the output layer during token prediction to transform the model's raw score outputs into probabilities over the vocabulary, enabling the selection of the most likely next token.

Additionally, softmax is used within the self-attention mechanism to normalize the attention weights. This normalization allows the model to weigh the importance of different input tokens effectively. Thus, softmax plays a crucial role in handling probabilities both in self-attention and final predictions.

Q28. What is the purpose of residual (skip) connections in Transformer model layers?

The main purpose of residual (skip) connections in Transformer layers is to allow the gradients to flow directly through the network during backpropagation. This helps to mitigate the vanishing gradient problem in deep networks, ensuring effective training.

Specifically, they enable the output of a layer to be Layer Input + Layer Output rather than just the Layer Output. This means the layer learns an additive change rather than a new representation entirely. This facilitates training very deep architectures, leading to better overall performance.

Q29. Why is layer normalization used, and where is it applied in Transformer?

Layer normalization is used in Transformer to stabilize and accelerate training by normalizing the inputs across features within each layer, ensuring that activations have a mean of zero and variance of one. This normalization helps control the scale of gradients during backpropagation, preventing issues like exploding or vanishing gradients.

In Transformer architectures, layer normalization is applied before the self-attention and feedforward sublayers (Pre-LN) or after these sublayers within the residual connections (Post-LN). The Pre-LN variant improves gradient stability and allows faster

training without requiring learning rate warm-up, while Post-LN was used in the original Transformer but can lead to unstable gradients if not carefully managed.

Q30. What is cross-entropy loss, and how is it applied during transformer training?

Cross-entropy loss measures the difference between the predicted probability distribution of a model and the true distribution of target labels. In Transformer training, it quantifies how well the model predicts the next token by comparing its output logits (after softmax) with the actual token index.

The loss penalizes incorrect predictions with higher values, encouraging the model to assign greater probability to the correct token. Minimizing the cross-entropy loss through backpropagation helps optimize the model's parameters for accurate sequence generation.

Q31. Compare transformers and RNNs in terms of handling long-range dependencies.

Transformers are fundamentally better than Recurrent Neural Networks (RNNs) at capturing long-range dependencies due to their core architectural difference. RNNs process sequences sequentially and suffer from the vanishing gradient problem, which causes information from earlier steps to diminish over long sequences, making it difficult to relate distant tokens.

In contrast, Transformers utilize a self-attention mechanism that computes a direct relationship score between every pair of tokens in the sequence. This allows it to capture dependencies regardless of the distance between the tokens in $O(1)$ sequential steps, unlike the $O(n)$ steps required by RNNs. This non-sequential, parallelized attention allows Transformers to effectively model context across much longer spans.

Q32. What are the fundamental limitations of the Transformer model?

The fundamental limitations of the Transformer model include its high computational and memory demands. This is because of the quadratic complexity of the self-attention mechanism, which makes processing very long sequences challenging.

Additionally, they require vast amounts of diverse training data and can be sensitive to biased or low-quality data, impacting generalization and robustness. Despite these challenges, they remain powerful and better than traditional deep learning models like CNNs and RNNs.

Q33. How do transformers address the limitations of traditional deep learning models like CNNs and RNNs?

Transformers address the limitations of traditional deep learning models like CNNs and RNNs primarily through their self-attention mechanism and parallel processing capability. RNNs process data sequentially and struggle with long-range dependencies due to vanishing gradients. CNNs excel at local feature extraction but lack the ability to model sequential or long-range relationships across data.

The self-attention mechanism allows transformers to process all parts of the input in parallel, assessing the importance of every word/token to every other word/token regardless of their distance. Compared to traditional models, transformers can scale better and train faster on large datasets.

Q34. How do Transformer model address the vanishing gradient problem?

Transformers address the vanishing gradient problem primarily through the use of residual (skip) connections and layer normalization. Residual connections allow gradients to flow directly through the network by adding the input of a layer to its output, preventing gradients from shrinking during backpropagation.

Layer normalization helps stabilize training by keeping activations within a consistent range, which controls the scale of gradients. Additionally, the self-attention mechanism enables direct information flow between tokens, reducing the depth of dependency and further mitigating gradient vanishing.

Q35. What is the purpose of the position-wise feed-forward sublayer in the Transformer model?

The position-wise feed-forward sublayer in the Transformer model serves to independently process each position's representation in the sequence after the attention

mechanism. It consists of two linear transformations with a ReLU activation in between, applied identically and separately to each token vector.

This sublayer adds non-linearity and increases the model's capacity to learn complex features by transforming the attention output into richer representations for the next layers.

Q36. Can you briefly explain the difference between LLM training and LLM inference?

LLM training is the process where a large language model learns patterns and relationships from massive datasets, adjusting its internal parameters (weights) to minimize prediction error. LLM inference is the deployment phase where the trained, fixed-parameter model uses its learned knowledge to generate a response for a new, unseen input prompt. Training is computationally intensive and done once, while inference is fast and happens every time the model is used.

Q37. What is latency in LLM inference, and why is it important?

Latency in LLM inference refers to the time delay between submitting an input prompt to the model and receiving the final, complete output. It measures how quickly the model processes input and generates output, typically in milliseconds or seconds. It is a critical performance metric because it directly impacts the user experience and the throughput of applications.

Low latency is crucial for delivering smooth, real-time experiences in chatbots, coding assistants, or search systems. High latency can lead to slow, frustrating interactions, especially in real-time or conversational systems.

Q38. What is batch inference, and how does it differ from single-query inference?

In batch inference, LLMs process and generate outputs for a large set of accumulated input data (the "batch") all at once. Batch inference is highly efficient for offline or asynchronous tasks like bulk document classification or generating recommendations overnight. This contrasts with single-query inference (or real-time inference), where the model processes individual data points as they arrive, providing responses with very low latency.

Single-query inference is typically required for interactive, user-facing applications like chatbots. The key difference lies in latency requirements and throughput optimization: batch inference prioritizes high throughput (processing a lot of data quickly) while sacrificing low latency, whereas single-query inference prioritizes minimal latency.

Q39. How does batching generally help with LLM inference efficiency?

Batching significantly improves LLM inference efficiency by enabling parallel processing of multiple requests, which maximizes GPU utilization. Instead of processing each query sequentially, batching groups requests together to leverage the full compute capacity of hardware, leading to higher throughput (tokens-per-second).

Continuous batching further enhances efficiency by using iteration-level scheduling, where new requests can replace completed ones within a batch without waiting for all sequences to finish, achieving much higher throughput improvements. This approach transforms GPU utilization from underutilized sequential processing to optimized parallel execution, making it essential for production LLM serving.

Q40. Explain the trade-offs between batching and latency in LLM serving.

Batching in LLM serving increases throughput by processing multiple requests concurrently, maximizing GPU utilization and efficiency, especially under heavy workloads. However, larger batches inevitably increase latency since faster or shorter requests must wait for the longest sequence to complete before the batch finishes.

Continuous or dynamic batching strategies attempt to balance this by adjusting batches in real time, replacing completed requests with new ones to maintain efficiency while minimizing waiting time. Ultimately, the trade-off depends on application needs—interactive systems prioritize low latency, whereas industrial or offline jobs aim for higher throughput at some latency cost.

Q41. How can techniques like mixture-of-experts (MoE) optimize inference efficiency?

Mixture-of-Experts (MoE) optimizes inference efficiency by introducing conditional computation or sparsity. Instead of activating all model parameters for every input token, a lightweight gating network dynamically selects only a small subset of specialized "expert" sub-networks to process the token.

This selective activation means the model performs significantly fewer floating-point operations (FLOPs) per token compared to a dense model of a comparable total size. This results in faster token generation (reduced latency) and higher throughput.

Q42. Explain the role of decoding strategy in LLM text generation.

Decoding strategies in LLMs are the decision rules a model uses to choose the next token from its vocabulary when generating text. An LLM doesn't just "know" the next word; it calculates a probability for every single possible word it knows. The decoding strategy is simply the method used to select the final word from that list of probabilities to form the output text, one word at a time.

The choice of strategy heavily influences whether the output is predictable and factual, or creative and diverse. Common decoding strategies include greedy search, beam search, and sampling methods like temperature scaling and top-k/top-p sampling.

[LLM Engineer Toolkit](#)

 This repository contains a curated list of 120+ LLM libraries category wise.

 **LLM Engineer Toolkit**

This repository contains a curated list of 120+ LLM libraries category wise.

 Kalyan KS  Kalyan KS  Kalyan KS

Quick links

 LLM Training	 LLM Application Development	 LLM RAG
 LLM Inference	 LLM Serving	 LLM Data Extraction
 LLM Data Generation	 LLM Agents	 LLM Evaluation
 LLM Monitoring	 LLM Prompts	 LLM Structured Outputs
 LLM Safety and Security	 LLM Embedding Models	 Others

This repository is highly useful for AI/ML Engineers. 👉 [Visit the repo](#)

Q43. What are the different decoding strategies in LLMs?

Decoding strategies in LLMs determine how tokens are selected during text generation. The main strategies are:

Greedy Decoding - Always picks the token with the highest probability at each step. It's fast but can produce repetitive or suboptimal text.

Beam Search - Keeps multiple best candidate sequences ("beams") at each step and chooses the most likely overall sequence. It improves quality but increases computation.

Top-k Sampling - Randomly samples the next token from the top k most probable tokens, adding diversity and reducing repetition.

Top-p (Nucleus) Sampling - Randomly samples from the smallest set of tokens whose cumulative probability $\geq p$ (e.g., 0.9).

Temperature Sampling - Adjusts randomness by scaling logits before softmax — higher temperature (>1) makes outputs more random; lower (<1) makes them more deterministic.

Speculative Decoding - Uses a smaller draft model to predict multiple tokens ahead and quickly verifies them with the main model, greatly speeding up generation without losing quality.

Q44. Explain the impact of the decoding strategy on LLM-generated output quality and latency.

Decoding strategies critically impact LLM output by balancing quality (coherence, diversity, and relevance) and latency (generation speed). Deterministic methods like Greedy Search are fast due to selecting the highest probability token at each step but often yield repetitive, lower-quality text. Beam Search improves quality by exploring multiple token sequences but increases latency due to managing several beams.

Stochastic decoding methods, such as Top-k or Top-p (nucleus) sampling, generate text token by token, choosing each next token probabilistically rather than deterministically. While this increases creativity and diversity, it also introduces extra computational overhead (sorting or cumulative sum) at every decoding step, which increases latency.

Newer, faster techniques like Speculative Decoding reduce latency by using a smaller model to draft tokens, which the larger model verifies in parallel, offering significant speedups while aiming to preserve the high output quality of the original model.

Q45. Explain the greedy search decoding strategy and its main drawback.

The greedy search decoding strategy in LLMs is a straightforward method where the model selects the token with the highest probability as the next word at each step of the generation process. This approach is highly efficient and deterministic, always producing the same output for a given input. Its primary drawback, however, is its shortsightedness.

By only focusing on the locally optimal choice, it often fails to find a sequence with the globally highest overall probability, potentially leading to repetitive, less coherent, or suboptimal text.

Q46. How does Beam Search improve upon Greedy Search, and what is the role of the beam width parameter?

Beam Search improves upon Greedy Search by exploring a wider set of possibilities. It keeps track of the k most probable partial sequences (where k is the beam width) at each decoding step, instead of just the single most probable one like Greedy Search. This significantly increases the chances of finding a globally better sequence by mitigating the risk of getting stuck with locally optimal, yet ultimately suboptimal, early choices.

The beam width (k) parameter determines the number of hypotheses maintained; a larger beam width explores more options and generally yields better results at the cost of higher computational complexity and slower decoding speed.

Q47. When is a deterministic strategy (like Beam Search) preferable to a stochastic (sampling) strategy? Provide a specific use case.

A deterministic strategy like Beam Search is preferable when consistency, reliability, and reproducibility of outputs are crucial. It systematically explores the most probable token sequences, making it ideal for tasks such as machine translation, summarization, or code generation where accuracy and coherence outweigh diversity.

For example, in legal or medical summarization, Beam Search ensures factual consistency and avoids random variations that sampling-based methods might introduce.

Q48. Discuss the primary trade-off between the computational cost and the output quality when comparing Greedy Search and Beam Search.

Greedy Search is computationally cheap because it only considers the single best token at each step, resulting in a fast decoding time with minimal memory use. However, it often leads to a sub-optimal output sequence because it cannot recover from locally good, but globally poor, choices.

Conversely, Beam Search maintains k (the beam width) of the most promising partial sequences at each step, significantly increasing the computational cost and decoding time and memory usage. However, this wider exploration of the search space dramatically improves the probability of finding a higher-quality, more globally optimal sequence.

Q49. When you set the temperature to 0.0, which decoding strategy are you using?

Setting the temperature to 0.0 means you're employing greedy decoding. This strategy forces the model to always select the token with the absolute highest probability at each step, making the output deterministic. It essentially eliminates all randomness, favoring the most probable sequence according to the model's learned distribution.

Q50. How is Beam Search fundamentally different from a Breadth-First Search (BFS) or Depth-First Search (DFS)?

Beam Search differs from Breadth-First Search (BFS) and Depth-First Search (DFS) in that it's not an exhaustive search algorithm but a heuristic-based, approximate search. Unlike BFS, which explores all nodes level-by-level, or DFS, which explores one path as deeply as possible, Beam Search keeps only the top- k most promising candidates at each step.

This greedy, constrained exploration makes it significantly more memory and time efficient than exhaustive BFS or DFS, which is crucial for large state spaces. However, it sacrifices completeness and optimality, as the best solution may be pruned.

Q51. Explain the criteria for choosing different decoding strategies.

The criteria for choosing different decoding strategies, such as greedy search, beam search, or sampling methods (like nucleus or top-k sampling), primarily depend on the desired balance between output quality, diversity, and computational cost. Greedy search is fast but often produces suboptimal, repetitive, or bland text, making it suitable when speed is critical and high quality is not paramount.

Beam search improves quality by exploring multiple paths and is preferred for tasks requiring deterministic outputs, like translation or summarization, though it's slower. Sampling methods are chosen when the goal is to generate more creative and diverse, less deterministic outputs, as in dialogue systems or story generation, by introducing controlled randomness.

Q52. Compare deterministic and stochastic decoding methods in LLMs.

Deterministic decoding methods, like Greedy Search and Beam Search, consistently yield the same output for the same input because they select the next token based purely on the highest probability or a fixed set of high-probability paths, respectively, without any element of randomness.

In contrast, stochastic decoding methods, such as Temperature Sampling, Top-K Sampling, and Top-p (Nucleus) Sampling, introduce randomness to the token selection process, resulting in varied and diverse outputs even for identical inputs.

Deterministic methods are generally preferred for tasks requiring predictability and factual accuracy (e.g., translation, summarization), while stochastic methods excel in open-ended or creative tasks (e.g., storytelling, brainstorming) where novelty is desirable.

Q53. What is the role of the context window during LLM inference?

The context window refers to the maximum total number of tokens (both input and output) that the model can process at one time. The context window during LLM inference serves as the model's working memory. This window enables the model to consider both the current input and preceding tokens to generate contextually relevant and coherent responses.

A larger context window allows the LLM to handle longer texts or complex tasks by preserving more information. However, if the input surpasses the window size, the model "forgets" earlier tokens, potentially reducing response accuracy. Essentially, it determines how much past information the model can use to make the next prediction.

Q54. Explain the pros and cons of large and small context windows in LLM inference.

The context window in LLMs defines the maximum number of tokens it can process at once, and it is like the model's working memory. A large context window allows the model to process more input, leading to more coherent and contextually aware outputs. However, processing more input tokens significantly increases memory usage, computational cost, and latency.

Conversely, a small context window is faster and more memory-efficient, making it suitable for lower-resource environments or real-time applications. However, it limits the model's 'memory,' potentially causing it to lose track of earlier information and generate less consistent or relevant outputs.

Q55. What is the purpose of temperature in LLM inference, and how does it affect the output?

The temperature parameter in LLM inference controls the randomness of the token selection process by rescaling the logits before the softmax function. Its purpose is to manage the trade-off between creativity and determinism in the generated output.

A high temperature (e.g., closer to 1.0) makes the probability distribution flatter, increasing the chance of selecting less likely tokens, resulting in more diverse, creative, and sometimes less coherent text. Conversely, a low temperature (e.g., closer to 0.0) makes the model more deterministic and focused on the most probable tokens, generating more coherent text.

Q56. What is autoregressive generation in the context of LLMs?

Autoregressive generation is the sequential process by which LLMs generate text, one token (word or sub-word unit) at a time, using the previously generated tokens and the original input as context. Essentially, the model predicts the next most probable token

based on all the preceding tokens, creating a dependency chain that results in coherent and contextually relevant text output. This process continues until an end-of-sequence token is predicted or a pre-defined length limit is reached.

Q57. Explain the strengths and limitations of autoregressive text generation in LLMs.

The main strength of autoregressive text generation in LLMs is its ability to produce coherent, contextually relevant, and high-quality sequential text. This happens because of its method of predicting the next token based on all preceding ones.

However, its primary limitation is that it's inherently slow because each new token must be generated serially, precluding true parallelization. Additionally, in the sequential generation process, errors can accumulate over time since each prediction depends on the previous tokens.

Q58. Explain how diffusion language models (DLMs) differ from Large Language Models (LLMs).

Diffusion Language Models (DLMs) differ from traditional Large Language Models (LLMs) in how they generate text. LLMs use autoregressive generation, predicting one token at a time based on previous ones. DLMs use a denoising process, starting from random noise and iteratively refining it into coherent text—similar to how diffusion models generate images. This approach allows DLMs to capture global context more effectively and potentially produce more diverse outputs.

Q59. Do you prefer DLMs or LLMs for latency-sensitive applications?

LLMs are autoregressive, generating text sequentially token-by-token, which creates a sequential bottleneck resulting in slower overall latency for long outputs. In contrast, DLMs are non-autoregressive and generate text by iteratively refining the entire sequence in parallel, which often offers them a significant advantage in inference speed and throughput for bulk processing or longer generations.

While a single denoising step in a DLM can be computationally heavier than an LLM's single token prediction, the ability to generate multiple tokens simultaneously over a few steps means that DLMs can achieve a faster Time Per Output Token (TPOT), making them an emerging alternative for latency-sensitive applications.

Q60. Explain the concept of token streaming during inference.

Token streaming during LLM inference is an optimization technique where the model's output, typically the next predicted token, is sent to the user immediately as soon as it's generated, rather than waiting for the entire response to be completed.

Token streaming significantly reduces the perceived latency because the user can begin reading the output almost instantly, making the interaction feel much faster and more responsive. The full response is thus "streamed" out token by token until an end-of-sequence token is reached.

Q61. What is speculative decoding, and when would you use it?

Speculative decoding accelerates LLM inference by pairing a smaller, faster "draft" model with a larger "target" model. The draft model speculatively generates multiple future tokens ahead of time, which the target model then verifies in parallel, accepting those matching its own predictions and correcting others.

This draft-then-verify approach reduces the sequential bottleneck of generating tokens one-by-one, improving GPU utilization and decreasing latency without sacrificing output quality. It is particularly useful in latency-sensitive applications like chatbots and code completion, where both speed and accuracy are critical.

Q62. What are the challenges in performing distributed inference across multiple GPUs?

The main challenges in performing distributed inference across multiple GPUs include memory management, communication overhead, workload balancing, and phase-specific resource needs.

1. Memory management is crucial because large models often do not fit into a single GPU, requiring model partitioning or sharding across GPUs.
2. Communication overhead arises from the synchronization of parameters and intermediate data between GPUs, which can add significant latency.
3. Workload balancing is needed to ensure that no single GPU becomes a bottleneck while others are underutilized, requiring effective parallelism strategies.

4. Lastly, different phases of inference, such as prefill (compute-bound) and decode (memory-bound), demand distinct GPU resources, complicating efficient resource allocation and orchestration.

These challenges demand careful orchestration and optimization to maximize throughput and minimize latency in multi-GPU distributed inference systems.

Q63. How would you design a scalable LLM inference system for real-time applications?

A scalable LLM inference system for real-time applications should use model sharding and distributed serving frameworks like vLLM to parallelize inference across multiple GPUs or nodes. The system should implement request batching, dynamic load balancing, and asynchronous processing to optimize GPU utilization and reduce latency.

Caching frequent prompts or embeddings further speeds responses, while autoscaling policies ensure resource efficiency during traffic spikes. Incorporating quantization and distillation can reduce model size and improve real-time performance without major accuracy loss.

Q64. Explain the role of flash attention in reducing memory bottlenecks during inference.

Flash Attention plays a crucial role in reducing memory bottlenecks during inference by optimizing the way the models handle attention computations. Traditional attention mechanisms incur high memory overhead due to frequent data transfers between slower high bandwidth memory (HBM) and faster but smaller on-chip SRAM, repeatedly loading and writing keys, queries, and values for each step.

Flash Attention significantly reduces memory bottlenecks during LLM inference by moving the computation of the large, intermediate attention scores matrix ($Q \cdot K^T$) from the slow High Bandwidth Memory (HBM) to the faster, on-chip SRAM. It achieves this by using a technique called tiling, where the attention calculation is broken into smaller blocks and computed incrementally, meaning the full, massive attention matrix is never explicitly materialized in the slower HBM.

This approach greatly decreases memory access latency and reduces computational overhead, enabling faster and more memory-efficient inference, especially beneficial for long input sequences.

Q65. What is continuous batching, and how does it differ from static batching?

Static batching involves grouping a fixed number of requests together and processing them simultaneously. Its main drawback is poor efficiency, as all requests must wait for the single longest sequence to finish, resulting in idle GPU time and increased latency. Continuous batching is a superior, dynamic technique that operates at the token generation level, immediately replacing a completed request with a new one.

This key difference ensures the GPU is constantly utilized, dramatically boosting overall throughput and reducing latency. Static batching is often preferred in offline scenarios where latency is less important, while continuous batching shines in online, interactive applications.

Q66. What is mixed precision (e.g., FP16) and why is it used during inference?

Mixed precision is a technique that uses a combination of different numerical formats, typically using the 16-bit floating-point format (FP16) for most computations, alongside higher-precision formats like FP32 where necessary for numerical stability. It is used during inference to significantly reduce both memory consumption and computational time.

Halving the bit-width cuts the model's memory consumption by roughly half, which allows for either larger models to fit onto the GPU or for a larger batch size. Crucially, modern hardware like NVIDIA Tensor Cores can execute FP16 operations significantly faster, thus boosting overall throughput with minimal loss in model accuracy.

Q67. Differentiate between online and offline LLM inference deployment scenarios and discuss their respective requirements.

Online LLM inference involves real-time, user-facing requests typically hosted on a cloud server, requiring low latency and high throughput to handle unpredictable traffic and network communication efficiently. Conversely, offline LLM inference deals with precollected data in batches, usually on on-premise or local hardware, where the primary requirements are high throughput and processing large data volumes at scale, with less stringent latency demands. The online scenario prioritizes rapid individual

responses, while the offline scenario focuses on massive-scale, non-real-time data processing.

Q68. Explain the throughput vs. latency tradeoff in LLM inference.

Latency refers to how long it takes to process a single request - the faster it responds, the lower the latency. This is usually achieved by handling smaller batches of data at a time. On the other hand, throughput measures how many requests a system can handle per second, which improves when larger batches are processed together to fully utilize the GPU.

However, doing so makes individual requests wait longer, increasing latency. Hence, systems must balance these metrics based on their application - interactive applications like chatbots focus on low latency for quick responses, while batch-processing systems aim for high throughput to maximize efficiency.

Q69. What are the various bottlenecks in a typical LLM inference pipeline when running on a modern GPU?

When running large language models (LLMs) on modern GPUs, several key bottlenecks limit performance. One major issue is memory bandwidth saturation, where the model frequently accesses large key-value (KV) caches, slowing data movement.

As the KV cache grows during text generation, it consumes more GPU memory, forcing smaller batch sizes and creating memory pressure. Compute bottlenecks also occur in heavy operations like matrix multiplications, though these are often less critical than memory-related delays.

In hybrid CPU-GPU systems, inefficient task scheduling can leave GPU cores underutilized, while multi-GPU setups face extra communication delays that reduce scalability. Overcoming these challenges involves techniques like caching, model quantization, smarter scheduling, and balanced workload distribution to fully harness GPU power and minimize latency.

Q70. How do you measure LLM inference performance?

LLM inference performance is primarily measured using latency and throughput metrics to gauge the model's speed and efficiency under load. Key metrics include Time

To First Token (TTFT), which measures how long it takes for the model to produce the first token after receiving a prompt, impacting user experience in real-time applications.

Time Per Output Token (TPOT) assesses the average time to generate each subsequent token, influencing the smoothness of the output stream. Overall latency is the total time from input submission to the complete response.

Throughput, another crucial metric, measures how many tokens or requests the system can handle per unit time, indicating its scalability. These metrics together help assess how fast, responsive, and scalable an LLM is in practical deployment scenarios.

Q71. What are the different LLM inference engines available? Which one do you prefer?

The most prominent LLM inference engines today are

- vLLM, which excels in high-throughput and memory efficiency via PagedAttention and continuous batching;
- NVIDIA TensorRT-LLM, which offers peak performance (lowest latency) by optimizing specifically for NVIDIA GPUs with custom CUDA kernels; and
- Hugging Face Text Generation Inference (TGI), a robust, production-ready solution well-integrated with the Hugging Face ecosystem.
- Other engines include LMDeploy and llama.cpp (for CPU/edge devices).

My preference leans towards vLLM due to its excellent balance of high throughput, ease of use (Hugging Face compatibility), and good hardware flexibility. These features make vLLM ideal for most scalable cloud-based serving environments.

Q72. What are the challenges in LLM inference?

The main challenges in LLM inference are high latency, computational intensity, memory constraints, token limits, accuracy issues including hallucinations, and scalability concerns.

1. High latency occurs because LLMs generate output token-by-token, creating delays in real-time applications.
2. Computational intensity means that running LLMs requires powerful and expensive hardware, leading to high operational costs.
3. Memory constraints limit the deployment of LLMs on devices with restricted memory capacity.

4. Token limits restrict input size, often necessitating truncation that can reduce context understanding.
5. Accuracy issues such as hallucinations can compromise output reliability.
6. Scalability remains a challenge in handling many concurrent requests without performance degradation.

Q73. What are the possible options for accelerating LLM inference?

Possible options to accelerate LLM inference include:

- Quantization: Reduces numerical precision to lower computation and memory use, speeding up inference.
- Pruning: Eliminates less important neurons or weights to make the model smaller and faster.
- Knowledge Distillation: Transfers knowledge to a smaller, more efficient model that runs faster.
- KV Caching: Stores and reuses previous attention computations to speed up token generation.
- Speculative Decoding: Uses a smaller draft model to generate candidate tokens verified by the full model.
- Hardware Acceleration: Utilizes GPUs, TPUs, or custom accelerators designed for parallel matrix operations.

Q74. What is Chain-of-Thought (CoT) prompting, and when is it most useful?

Chain-of-Thought (CoT) prompting is a technique where you instruct an LLM to explicitly show the step-by-step reasoning process before arriving at the final answer. This process mimics human-like thinking, breaking down complex problems into manageable intermediate steps.

CoT is most useful for complex reasoning tasks that require multiple steps, such as multi-step arithmetic, symbolic reasoning, and common-sense question answering. The

CoT prompting technique significantly improves the LLM's accuracy and ability to handle complexity compared to standard prompting.

Q75. Explain the reason behind the effectiveness of Chain-of-Thought (CoT) prompting.

The power of Chain-of-Thought (CoT) prompting lies in how it helps large language models think more like humans - step by step. Instead of jumping straight to an answer, CoT encourages the model to reason through the problem by breaking it down into smaller, logical steps.

This structured thinking makes it easier for the model to handle tasks involving logic, math, or common sense, resulting in more accurate and trustworthy answers. In simple terms, CoT gives the model a bit of extra “thinking time” before it decides on the final response.

Q76. Explain the trade-offs in using CoT prompting.

Chain-of-Thought (CoT) prompting improves accuracy and interpretability by encouraging models to generate intermediate steps before producing the final answer. However, it introduces trade-offs such as increased latency and token usage, which raise computational and cost overheads. Moreover, CoT can sometimes amplify hallucinations if the reasoning path is incorrect. It also requires a careful prompt design to balance reasoning detail with brevity for optimal performance.

Q77. What is prompt engineering, and why is it important for LLMs?

Prompt engineering is the process of designing and refining input prompts to effectively guide LLMs toward generating accurate, relevant, and coherent responses. Since LLMs interpret user intent based on textual cues, well-crafted prompts reduce ambiguity and improve performance. It is especially important because LLMs are sensitive to phrasing, context, and examples provided in the prompt. Effective prompt engineering enhances output quality without retraining the model, making it a key skill in leveraging LLMs efficiently for diverse applications.

Q78. What is the difference between zero-shot and few-shot prompting?

The fundamental difference lies in the number of examples within the prompt. In zero-shot prompting, the model is given only the instruction and input data without examples. Here the model completely depends on its pre-trained knowledge to generate the correct output. In contrast, few-shot prompting includes a small number of input-output examples (the "shots") in the prompt. The examples guide the LLM to understand the desired format, style, or task better, often leading to significantly improved performance.

Q79. What are the different approaches for choosing examples for few-shot prompting?

Proper choice of examples in few-shot prompting aims to maximize the informativeness and relevance of the limited context provided to the LLM. One common approach is random sampling, where examples are chosen arbitrarily from the dataset to provide a general overview.

Diversity-based selection ensures the examples cover a wide range of input scenarios and edge cases relevant to the task, preventing overfitting. Conversely, similarity-based selection (often using vector embeddings) retrieves examples semantically most similar to the input query, providing the most direct in-context guidance.

Q80. Why is context length important when designing prompts for LLMs?

Context length is crucial because it dictates the maximum number of tokens an LLM can consider when generating a response. If the prompt having instructions, examples, and input data exceeds this limit, the model will truncate the input.

This leads to a loss of vital information, which results in incomplete, irrelevant, or inaccurate outputs. Therefore, understanding and managing the context length is essential for designing concise yet comprehensive prompts that fully leverage the model's capabilities.

Q81. What is a system prompt, and how does it differ from a user prompt?

A system prompt is an instruction given to an LLM to define its overall behavior, role, or response style throughout a conversation, such as "You are an expert data scientist." It sets the foundation for how the model interprets and responds to inputs.

In contrast, a user prompt is a direct query or task provided by the user during interaction, like “Explain the concept of embeddings.” In simple words, the system prompt defines the model’s persona and boundaries, while the user prompt provides the query to be answered or the task to be performed.

Q82. What is In-Context Learning (ICL), and how is few-shot prompting related?

In-Context Learning (ICL) is a powerful ability of large language models that lets them perform tasks simply by understanding the input prompt. Few-shot prompting is a specific type of ICL where a few labeled examples are added to the prompt. The added examples show the model how the task should be done, helping it generalize to similar inputs. ICL allows LLMs to quickly adapt to new tasks with minimal data and no retraining.

Q83. What is self-consistency prompting, and how does it improve reasoning?

Self-consistency prompting is a technique in LLMs where the model generates multiple reasoning paths or answers for the same input and then aggregates them to select the most consistent solution. Instead of relying on a single output, it considers the agreement among several reasoning attempts, which reduces the likelihood of errors caused by spurious or biased reasoning steps.

This approach improves reasoning by amplifying correct patterns and filtering out inconsistent or unlikely answers, which results in more accurate outputs.

Q84. Why is context important in designing prompts?

Context is crucial in prompt design because it provides the LLMs with the necessary background, constraints, and specific role it should adopt. This helps LLMs to generate a relevant and high-quality output. Without sufficient context, the LLM may misinterpret the user’s intent and rely on general knowledge. This may result in the generation of ambiguous, generic, or incorrect responses that don't meet the user's specific needs.

Q85. Describe a strategy for reducing hallucinations via prompt design.

A key strategy for reducing hallucinations in LLMs via prompt design is grounding the response by instructing the model to rely exclusively on provided context. This is often achieved by including a clear directive such as: "Answer the following question only using the provided document. If the document does not contain the answer, state 'The information is not available in the document.' Do not use any external knowledge."

This constrains the model's search space to the provided input, significantly decreasing the likelihood of generating fabricated or incorrect details.

Q86. How would you structure a prompt to ensure the LLM output is in a specific format, like JSON?

To ensure an LLM output is in a specific format like JSON, you should include an explicit instruction in the prompt, clearly stating the desired output structure. For example, "Please respond with a valid JSON object."

It is also helpful to provide a schema or example of the required JSON structure, including the keys and expected data types for each value. Additionally, including an explicit instruction to only output the JSON and nothing else helps prevent explanation text from being included, resulting in a clean, parsable output.

Q87. Explain the purpose of ReAct prompting in AI Agents.

ReAct (Reasoning and Acting) prompting allows the agent to solve complex, multi-step tasks by allowing it to dynamically plan, execute external actions (e.g., using a search engine or tool), and refine its approach based on observations.

This loop of thinking, acting, and observing keeps the model's reasoning grounded in real-world feedback, which reduces hallucinations and makes its decisions more accurate, interpretable, and reliable. By combining logical reasoning with real-world interaction, ReAct enables more flexible, reliable, and human-like problem-solving.

Q88. What are the different phases in LLM development?

The typical development process for a Large Language Model (LLM) involves three primary phases:

- pre-training, where the model learns foundational language understanding from massive, general datasets through self-supervised tasks like predicting the next word;

- followed by instruction-tuning, where the pre-trained model is fine-tuned on diverse examples of instructions and corresponding desired outputs to make it better at following instructions and performing specific tasks;
- and finally, alignment tuning, which further refines the model's behavior to align with human values, preferences, and safety standards, resulting in more helpful and harmless responses.

Q89. What are the different types of LLM fine-tuning?

Fine-tuning, in the context of LLMs, is a **broad term** that can refer to **instruction fine-tuning**, **task-specific fine-tuning**, or **alignment tuning**.

- **Instruction fine-tuning** involves fine-tuning the model on a dataset of high-quality **(instruction, response)** pairs to improve its ability to follow instructions and generalize across various tasks.
- **Task-specific fine-tuning** involves fine-tuning the model on a dataset tailored to a **single, specific downstream application** (e.g., sentiment analysis, text summarization) to maximize performance on that particular task.
- **Alignment Tuning** involves fine-tuning the model using Reinforcement Learning (RL) to adjust the model's behavior to be safe, helpful, and align with human values and preferences.

Q90. What role does instruction tuning play in improving an LLM's usability?

Instruction tuning greatly improves how well a large language model (LLM) understands and follows user directions in the input prompt. While raw, pretrained LLMs trained only to predict the next word are good at continuing text. But they often struggle to follow explicit instructions.

Instruction fine-tuning fixes this by training the model on high-quality pairs of prompts and responses. Through exposure to diverse examples, the model learns to correctly interpret the user instructions and perform the given tasks. To summarize, instruction tuning makes LLMs better at understanding the user prompts and producing desired outputs.

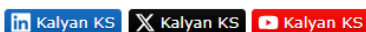
Q91. What role does alignment tuning play in improving an LLM's usability?

Alignment tuning enhances an LLM's usability by ensuring its responses align with human values and ethical principles, making interactions safer and more relatable. This process ensures the model is not only helpful (answering questions effectively) but also harmless (refusing to generate toxic or unethical content). By tuning the model to a preference reward signal, alignment makes the LLM's output feel more natural, safe, and trustworthy, fundamentally enhancing the user experience.

LLM Survey Papers Collection

LLM Survey Papers Collection

This repo contains a category wise collection of 200+ LLM survey papers.



Quick links

 General	 Prompting Techniques	 Small Language Models
 Multi Modal LLMs	 Multilingual LLMs	 Data Management
 Post Training	 LLM Fine Tuning	 LLM Inference
 LLM Architectures	 LLM Abilities and Limitations	 LLM Reasoning
 Chain of Thought	 LLM RAG	 LLM Agents
 Data Augmentation	 Hallucination	 Fairness
 Embedding Models	 LLM Evaluation	 Synthetic Data Generation
 LLM Safety	 LLMs for Specific Domains	 Others

This repo is highly useful to stay updated with research related to LLMs, RAG and Agents. 👉 [Visit the repo.](#)

Q92. How do you prevent overfitting during fine-tuning?

To prevent overfitting during LLM fine-tuning, the primary methods involve using a validation set to monitor performance and employing early stopping when the validation loss starts to increase, indicating the model is memorizing the training data. Additionally, techniques like regularization (e.g., L2 or dropout) can be applied to penalize complex models.

Lastly, ensuring the fine-tuning dataset is diverse and sufficiently large relative to the model size avoids overfitting and helps the model to generalize better to unseen data.

Q93. What is catastrophic forgetting, and why is it a concern in fine-tuning?

Catastrophic forgetting is the loss of previously learned capabilities or knowledge when an LLM is fine-tuned on new, distinct data. It occurs because fine-tuning often involves updating all or most of the model's parameters, causing the new training signal to drastically alter weights important for the old tasks. This is a significant concern because it compromises the general utility of the base LLM, meaning the model might excel at the new, fine-tuned task but become incapable of performing the original, broader set of tasks it was initially trained for.

Q94. What are the strengths and limitations of full fine-tuning?

Full fine-tuning involves updating all parameters of a pre-trained LLM. Its primary strength is achieving the highest possible performance because the model is fully adapted, potentially leading to state-of-the-art results. However, its major limitations include

- being computationally expensive and time-consuming, demanding significant GPU resources, large storage for the full model checkpoints, and
- a high risk of catastrophic forgetting, where the model loses its general knowledge acquired during pre-training.

Q95. Explain how parameter efficient fine-tuning addresses the limitations of full fine-tuning.

Parameter-efficient fine-tuning (PEFT) addresses the limitations of full fine-tuning by updating only a small subset of model parameters or adding lightweight modules

(introducing a minimal number of new, trainable parameters) while keeping the majority of the pre-trained model weights frozen. This approach significantly reduces computational costs, memory usage, and storage requirements compared to full fine-tuning, which updates all parameters and demands high resources. PEFT also mitigates the risk of catastrophic forgetting and overfitting that full fine-tuning faces by preserving the original pre-trained knowledge. Additionally, PEFT enables faster training and easier deployment on edge devices or resource-constrained environments, making it more scalable and cost-effective without sacrificing performance. This balance of efficiency and effectiveness makes PEFT a practical solution for adapting large language models across multiple domains and tasks.

Q96. When might prompt engineering be preferred over task-specific fine-tuning?

Prompt engineering is generally preferred over task-specific fine-tuning when the task is complex or open-ended, requiring the LLM's broad general knowledge and in-context learning abilities to solve it.

It is also the better choice when data for fine-tuning is scarce or if rapid iteration and experimentation are needed, as modifying a prompt is significantly faster and more resource-efficient than fine-tuning the model. Furthermore, if a single LLM must handle a variety of diverse tasks, prompt engineering is preferred, as it avoids creating and deploying a separate fine-tuned model for each.

Q97. When should you use fine-tuning vs. RAG?

Fine-tuning is best when you want the model to deeply learn domain-specific knowledge or handle specialized tasks where the knowledge is relatively static and not frequently changing. On the other hand, RAG (Retrieval-Augmented Generation) is ideal when you need the model to access the latest, proprietary, or frequently changing information without retraining it. It allows the model to provide fact-grounded answers and source traceability from a secure knowledge base.

Q98. Explain the limitations of using RAG over LLM fine-tuning.

The main drawbacks of using RAG compared to fine-tuning lie in its performance and lack of deep specialization. Because RAG adds an extra retrieval step, it introduces more inference latency, making it less ideal for low-latency use cases. While RAG is effective

at bringing in external knowledge, it doesn't actually change the model's core behavior, style, or ability to handle complex, domain-specific reasoning.

Fine-tuning achieves all these by updating the model's weights. Additionally, RAG's output quality heavily depends on the retriever's accuracy, meaning it can produce poor results if irrelevant information is fetched.

Q99. Explain the limitations of using LLM fine-tuning over RAG.

The main drawback of fine-tuning compared to RAG is the model's static knowledge. Once the model is trained, it can't access new or real-time information without undergoing an expensive and time-consuming retraining process. Fine-tuning also requires significant computational resources and specialized expertise for preparing data and training the model.

Moreover, fine-tuned models risk "catastrophic forgetting," where learning new information causes them to lose some of their original general knowledge. RAG avoids the catastrophic forgetting problem, as it keeps the base model unchanged.

Q100. When should you prefer task-specific fine-tuning over prompt engineering?

You should opt for task-specific fine-tuning when prompt engineering alone doesn't deliver the desired level of performance. This often happens in the case of complex, specialized tasks in fields like law or medicine, where the base model may lack the needed deep domain knowledge.

Fine-tuning is also ideal when you need low-latency, cost-efficient inference in large-scale production systems, as it produces a smaller and more optimized model than one relying on lengthy prompts. Lastly, fine-tuning offers greater control over the model's behavior, making it easier to ensure consistency, which prompting alone can't always guarantee.

Q101. What is LoRA, and how does it work at a high level?

LoRA (Low-Rank Adaptation) is a Parameter-Efficient Fine-Tuning (PEFT) technique that allows you to fine-tune LLMs by updating only a small number of additional parameters instead of modifying all the model's weights.

At a high level, LoRA works by injecting small trainable low-rank matrices into specific layers of the model (typically the attention or feedforward layers). Instead of updating the large weight matrix W , LoRA keeps W frozen and adds a low-rank decomposition $\Delta W = AB$, where A and B are much smaller matrices ($r \ll d$). During training, only A and B are learned, significantly reducing memory and compute costs.

At inference, the adapted weights are effectively merged with the original model weights, so the model behaves as if it was fully fine-tuned but with far fewer parameters trained. This makes LoRA both efficient (in storage and computation) and modular.

Q102. Explain the key ingredient behind the effectiveness of the LoRA technique.

The key ingredient behind the effectiveness of the LoRA technique lies in its low-rank decomposition of weight updates, which captures task-specific information within a much smaller subspace of the model's full parameter space. By expressing the weight change as the product of two small matrices, LoRA enables efficient fine-tuning with minimal memory and computational overhead.

This approach leverages the observation that large language models have redundant parameters, and most adaptations can be represented in a low-dimensional form. As a result, LoRA achieves performance comparable to full fine-tuning while training only a tiny fraction of the model's parameters.

Q103. What is QLoRA, and how does it differ from LoRA?

QLoRA (Quantized Low-Rank Adaptation) is an extension of LoRA that enables fine-tuning large language models more efficiently by combining low-rank adaptation with quantization. While LoRA freezes most model weights and trains small low-rank matrices to reduce memory and compute costs, QLoRA goes a step further by quantizing the model weights to 4-bit precision during fine-tuning.

This drastically lowers GPU memory requirements without significant performance loss, allowing fine-tuning of very large models on consumer-grade hardware.

Q104. When would you use QLoRA instead of standard LoRA?

QLoRA is preferred over standard LoRA when fine-tuning very large language models on limited hardware resources, such as a single GPU with constrained memory.

QLoRA's 4-bit quantization significantly reduces memory usage while maintaining model performance, making it ideal for resource-efficient fine-tuning. It's particularly useful when working with models like LLaMA that would otherwise exceed GPU limits. In contrast, standard LoRA is sufficient when hardware capacity is not a major constraint.

Q105. How would you handle LLM fine-tuning on consumer hardware with limited GPU memory?

When fine-tuning an LLM on consumer hardware with limited GPU memory, techniques like LoRA (Low-Rank Adaptation) or QLoRA can be used to reduce memory usage by training only a small subset of parameters. Additionally, using gradient accumulation, mixed precision training, smaller batch sizes, and smaller sequence lengths helps manage GPU constraints. These approaches minimize memory overhead and computational cost, making fine-tuning LLMs feasible on resource-constrained devices.

Q106. Explain different preference alignment methods and their trade-offs.

Different preference alignment methods in LLMs include Reinforcement Learning from Human Feedback (RLHF), Direct Preference Optimization (DPO), Odds Ratio Preference Optimization (ORPO), and Kahneman-Tversky Optimization (KTO).

1. RLHF uses a reward model trained on human-labeled examples to guide model behavior but is computationally expensive and complex.
2. DPO simplifies this by directly optimizing the model on preference data without a separate reward model, offering efficiency and stability but depending heavily on data quality.
3. ORPO combines task objectives and preference alignment in one loss, improving training efficiency but increasing implementation complexity.
4. KTO uses binary good/bad labels, is robust to noisy data, and is simple to label, yet it may lack granularity for nuanced alignment.

Trade-offs revolve around complexity, computational resources, data requirements, and alignment precision, with the optimal method chosen based on specific use cases and resource constraints.

Q107. What is gradient accumulation, and how does it help with fine-tuning large models?

Gradient accumulation is a technique used in fine-tuning large language models (LLMs) that helps manage GPU memory limitations by simulating larger batch sizes. Instead of updating model weights after processing each mini-batch, gradients are accumulated over several smaller batches, and the model parameters are updated once after the accumulation.

This approach enables training with effective large batch sizes even on memory-constrained hardware, improving the stability and performance of the fine-tuning process. It allows fine-tuning of LLMs on less powerful GPUs and reduces hardware cost while maintaining training quality.

Q108. What are the possible options to speed up LLM fine-tuning?

To speed up LLM fine-tuning, several optimization strategies can be used. Techniques like LoRA (Low-Rank Adaptation) or QLoRA (Quantized LoRA) reduce the number of trainable parameters, significantly lowering GPU memory usage and training time.

Additionally, mixed precision training (FP16/BF16) for faster computation, gradient accumulation for simulating larger batch sizes, and distributed training across multiple GPUs achieve quicker convergence and shorter training times. These approaches drastically cut down both computational cost and the fine-tuning time.

Q109. Explain the pretraining objective used in LLM pretraining.

The pre-training objective of large language models is next token prediction, also known as causal language modeling. In this setup, the model learns to predict the next token in a sequence given all the previous ones. By minimizing the difference between its predictions and the actual next tokens across billions of examples, the model gradually learns grammar, semantics, and contextual relationships. This objective enables LLMs to generate coherent, contextually relevant text and perform a wide range of downstream tasks through prompting or fine-tuning.

Q110. What is the difference between casual language modeling and masked language modeling?

Causal Language Modeling (CLM) is an autoregressive approach where the model predicts the next token in a sequence based only on the preceding tokens. In contrast, Masked Language Modeling (MLM) is an autoencoding approach where the model

predicts intentionally masked (missing) tokens by leveraging bidirectional context, i.e., it considers both the past and future tokens in the sequence.

Q111. How do LLMs handle out-of-vocabulary (OOV) words?

LLMs handle out-of-vocabulary (OOV) words using subword tokenization methods such as Byte Pair Encoding (BPE), WordPiece, or SentencePiece. These techniques split rare or unseen words into smaller, known subword units or characters. This allows the model to represent and understand new words from existing tokens in the vocabulary. For example, the word “unhappiness” might be split into “un”, “happy”, and “ness”. This approach reduces the OOV problem and improves generalization to unseen vocabulary.

Q112. In the context of LLM pretraining, what is scaling law?

In the context of LLM pretraining, scaling laws describe the predictable relationship between a model’s performance and its key factors—such as model size (number of parameters), dataset size, and compute resources. Empirical studies show that as these factors increase, model performance improves following a power-law trend until diminishing returns appear.

This law provides a crucial guide for efficiently designing and allocating resources for large-scale model training by predicting the optimal balance of data, parameters, and compute needed to achieve a target performance level.

Q113. Explain the concept of Mixture-of-Experts (MoE) architecture and its role in LLM pretraining.

The Mixture-of-Experts (MoE) architecture significantly improves Large Language Model (LLM) pretraining efficiency and capacity by replacing the standard feed-forward layers with a set of specialized "expert" networks. A "router" or "gating network" learns to selectively activate a small subset of these experts for each input token.

This allows the model to (i) dramatically increase its total parameter count and thus its capacity for knowledge and (ii) maintain a low computational cost during inference, as only a fraction of the parameters are used for any given input. This sparsity enables models with billions of parameters to be trained and run more efficiently, facilitating the scaling of LLMs to unprecedented sizes.

Q114. What is model parallelism, and how is it used in LLM pre-training?

Model parallelism is a technique used to train large language models that are too big to fit on a single GPU by splitting the model's parameters across multiple devices. Instead of each GPU holding a full model copy, different GPUs handle different layers or parts of the same layer.

During forward and backward passes, activations and gradients are communicated between GPUs to complete computation. This allows efficient utilization of hardware for massive models. However, it requires careful coordination to minimize communication overhead and latency.

Q115. What is the significance of self-supervised learning in LLM pretraining?

Self-supervised learning (SSL) is crucial for LLM pretraining because it allows models to learn rich, general-purpose language representations from massive amounts of unlabeled text data. It works by creating a surrogate task, like predicting the next word, where the "labels" are automatically derived from the input itself. This eliminates the need for expensive human annotation. By predicting the next token in a sequence, the model learns syntax, semantics, and world knowledge from massive unlabeled corpora.

AIXFunda Newsletter (free)

Want to beat FOMO in Generative AI? Then join  AIXFunda free newsletter to get the latest updates and interesting tutorials related to Generative AI, LLMs, Agents and RAG.

✨ Weekly GenAI updates.

 Weekly LLM, Agents and RAG paper updates.

 1 fresh blog post on an interesting topic every week.



The banner features a green background. At the top center is the AIXFunda logo, which includes a rocket icon and the text 'AIXFunda'. To the left of the logo is an envelope icon with an '@' symbol. To the right of the envelope icon is the text 'Join AIXFunda Newsletter for the latest updates related to Generative AI, LLMs, Agents and RAG.' Below this text is a circular profile picture of Kalyan KS, a man with glasses and a beard. Below the profile picture is the text 'Kalyan KS' and 'NLP Consultant & Researcher'. To the right of the profile picture is a blue button with the text 'Subscribe Now'. On the far right of the banner is a cartoon illustration of a man with glasses and a blue hoodie sitting cross-legged on the floor, working on a laptop. The laptop screen displays 'AIXFunda'. There are coffee cups and papers scattered around him. Above his head are thought bubbles containing a gear, a lightbulb, and a percentage sign.

👉 [Subscribe](#)



Kalyan KS

NLP Consultant & Researcher